Grid Search: Optimización de Hiperparámetros

Aprendizaje Supervisado · Validación Cruzada y GridSearchCV · Aprobación de Crédito

Inteligencia Artificial · Sexto Semestre · Ingeniería de Sistemas de Información · PUCE

Objetivos de Aprendizaje

Al finalizar esta clase serás capaz de:

1. Distinguir parámetros de hiperparámetros y explicar cómo funciona la validación cruzada k-fold.
2. Configurar GridSearchCV con un Pipeline, un param_grid y la métrica de scoring adecuada al problema.
3. Interpretar best_params_, best_score_ y cv_results_, y evaluar el mejor modelo en el conjunto de prueba.
4. Reentrenar el modelo final con el 100% de los datos, exportarlo con joblib y usarlo con datos nuevos.

Parámetros vs. Hiperparámetros

Todo modelo tiene dos tipos de valores: unos los aprende, otros los decidimos nosotros

Definición: un **hiperparámetro** es un valor de configuración del modelo que se fija **antes** de entrenar; `fit()` no lo aprende de los datos.

Parámetros (los aprende fit())

- Regresión Lineal: coeficientes β.
- Regresión Logística: `coef_` e `intercept_`.
- SVM: los vectores de soporte.
- En scikit-learn terminan en guion bajo: `coef_`.

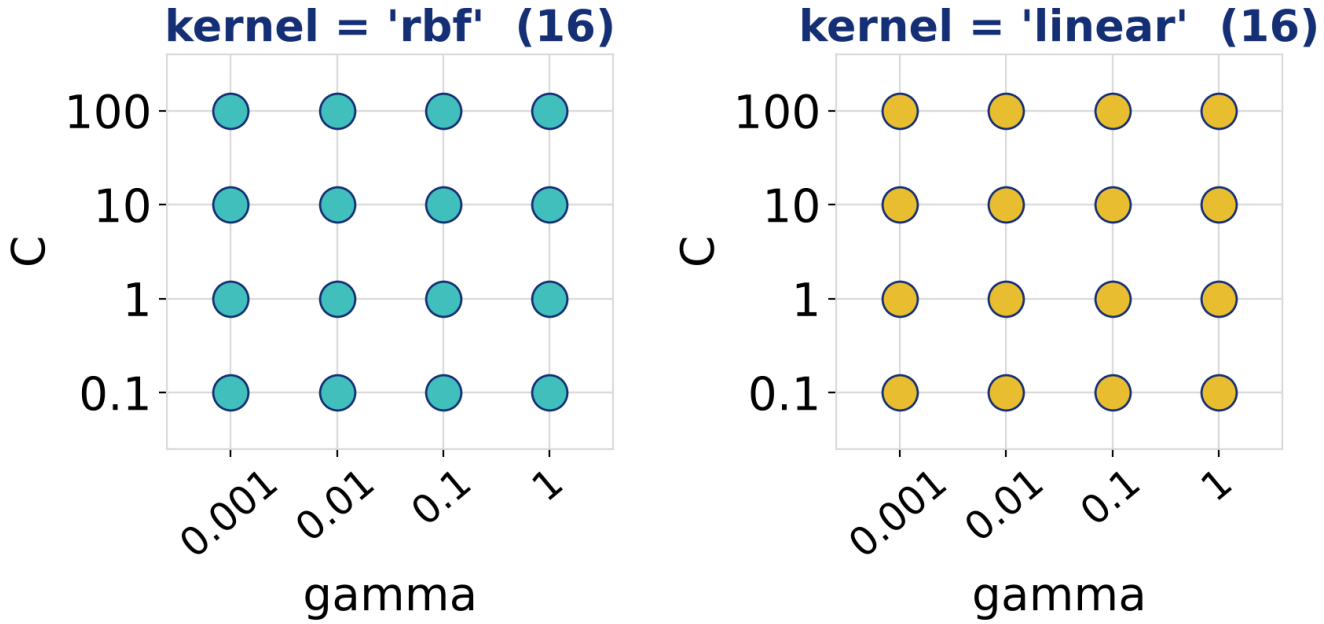
Hiperparámetros (los elegimos)

- SVC: `C`, `gamma`, `kernel`.
- LogisticRegression: `max_iter`, `C`.
- PolynomialFeatures: `degree`.
- Se pasan al constructor: `SVC(C=10)`.

Un mal valor de hiperparámetro arruina el modelo aunque los datos sean buenos: ¿cómo elegir el mejor de forma objetiva?

¿Por Qué no Elegir los Hiperparámetros a Mano?

Las combinaciones crecen muy rápido: cada punto es un modelo distinto



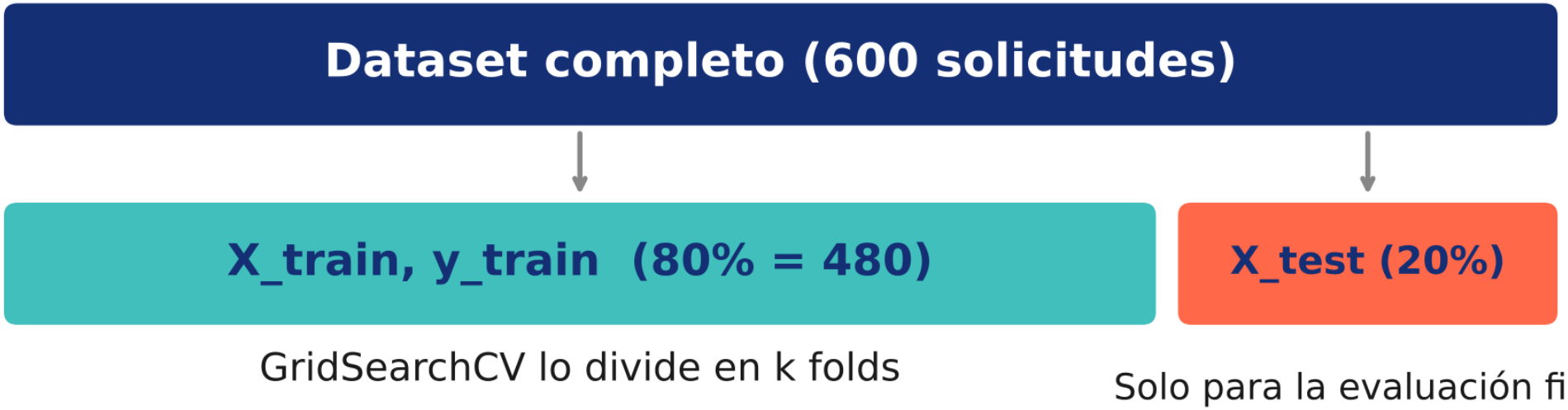
- Probar 32 modelos a mano es lento y propenso a errores.
- Elegir "a ojo" depende de la intuición, no de evidencia.
- Con más hiperparámetros, el número se multiplica.
- **GridSearchCV** prueba todas las combinaciones de forma automática y reproducible.

$$N_{\text{comb}} = n_C \times n_\gamma \times n_{\text{kernel}} = 4 \times 4 \times 2 = 32$$

Producto cartesiano: se combinan TODOS los valores de cada hiperparámetro

¿Con Qué Datos se Eligen los Hiperparámetros?

El conjunto de prueba no se toca hasta el final



✕ Elegir con X_test

- El modelo se ajusta a esos 120 casos.
- La métrica final sale optimista: ya no son datos "nuevos".

✓ Elegir con validación cruzada

- Se usan solo los datos de entrenamiento.
- X_test queda intacto para medir el rendimiento real.

Validación Cruzada k-fold

La base de GridSearchCV: evaluar cada combinación varias veces con datos distintos

Definición: divide X_train en **k partes (folds)**; entrena k veces, cada vez con k−1 folds, y valida con el fold restante.



Score final (k = 5)

$$\text{Score}_{CV} = \frac{1}{k}\sum_{i=1}^{k} \text{score}_i$$

- Cada dato valida exactamente una vez.
- Promediar 5 scores es más confiable que uno solo.
- En clasificación, `cv=5` mantiene la proporción de clases en cada fold.

GridSearchCV: Definición y Funcionamiento

GridSearchCV = Grid Search Cross-Validation (búsqueda en rejilla con validación cruzada)

Definición: clase de scikit-learn que prueba **todas** las combinaciones de un param_grid con validación cruzada y se queda con la mejor.

1	Recibe el modelo (Pipeline) y el param_grid.
2	Genera todas las combinaciones (producto cartesiano).
3	Evalúa cada combinación con k-fold → Score CV.
4	Elige la combinación con el Score CV más alto.
5	Reentrena esa combinación con todo X_train (refit=True).

¿Por Qué Usar un Pipeline Dentro de GridSearchCV?

El escalado también aprende de los datos (media y desviación): debe repetirse en cada fold

✕ Escalar antes de GridSearchCV

- `scaler.fit(X_train)` usa los 5 folds, también el de validación.
- El fold de validación ya "vio" su media y desviación.
- El Score CV sale optimista: **fuga de datos** (data leakage).

✓ Pipeline dentro de GridSearchCV

- En cada iteración el escalador se ajusta solo con los 4 folds de entrenamiento.
- El fold de validación se trata como datos nuevos.
- Es el mismo flujo que en producción.

```
pipe = Pipeline([('escalar', StandardScaler()),
                 ('modelo', SVC())])
```

Regla: todo paso que usa fit() va dentro del Pipeline

Nombres en el param_grid: paso__parámetro

Con un Pipeline hay que indicar a qué paso pertenece cada hiperparámetro

Nombre en el param_grid	Paso del Pipeline	Hiperparámetro
`'modelo__C'`	'modelo' → SVC()	C
`'modelo__gamma'`	'modelo' → SVC()	gamma
`'modelo__kernel'`	'modelo' → SVC()	kernel

- Se escribe: nombre del paso + **dos guiones bajos** `__` + nombre del hiperparámetro.
- El nombre del paso es el que pusimos en el Pipeline: `('modelo', SVC())`.

⚠️ **Error frecuente:** escribir `'C'` sin el prefijo. GridSearchCV busca C en el Pipeline, no en SVC(), y falla con: `ValueError: Invalid parameter 'C' for estimator Pipeline`.

Diseñar el param_grid

Diccionario: clave = hiperparámetro · valor = lista de opciones a probar

```
param_grid = {
    'modelo__C':      [0.1, 1, 10, 100],          # escala ×10
    'modelo__gamma':  [0.001, 0.01, 0.1, 1],      # escala ×10
    'modelo__kernel': ['rbf', 'linear'],
}
```

- **Escala logarítmica:** C y gamma actúan por órdenes de magnitud; conviene saltar ×10, no de 0.01 en 0.01.

Costo de la búsqueda (k = 5 folds)

$$N_{\text{comb}} = n_C \times n_\gamma \times n_{\text{kernel}} = 4 \times 4 \times 2 = 32$$

$$\text{Entrenamientos} = N_{\text{comb}} \times k \; + \; 1 = 32 \times 5 + 1 = 161$$

+1 = reentrenamiento final del mejor modelo con todo X_train (refit=True).

Combinaciones Repetidas: gamma y el Kernel Lineal

gamma solo afecta a los kernels 'rbf' y 'poly'; el kernel lineal lo ignora

Ejemplo real (diabetes_ecuador.csv): con el param_grid anterior, **4 combinaciones empatan** en el 1.er lugar: kernel lineal con C = 1 y cada uno de los 4 valores de gamma.

```
param_grid = [
    {'modelo__kernel': ['rbf'],
     'modelo__C': [0.1, 1, 10, 100],
     'modelo__gamma': [0.001, 0.01, 0.1, 1]},    # 16
    {'modelo__kernel': ['linear'],
     'modelo__C': [0.1, 1, 10, 100]},            # 4
]
```

Una lista de diccionarios evita combinaciones inútiles: 20 en vez de 32 (100 entrenamientos en vez de 160)

La Clase GridSearchCV() (1/2): Parámetros

Se pasan al crear el objeto · from sklearn.model_selection import GridSearchCV

Parámetro	Defecto	Qué controla
`estimator`	(obligatorio)	Modelo o Pipeline a optimizar
`param_grid`	(obligatorio)	Diccionario con las opciones a probar
`cv`	None → 5 folds	Número de folds de la validación cruzada
`scoring`	None → score()	Métrica a maximizar (en SVC: accuracy)
`n_jobs`	None → 1 núcleo	-1 = usar todos los núcleos del procesador
`refit`	True	Reentrena el mejor con todo X_train
`verbose`	0	Mensajes de progreso (0 = ninguno)

```
grid = GridSearchCV(pipe, param_grid, cv=5,
                    scoring='f1', n_jobs=-1)
```

La Clase GridSearchCV() (2/2): Métodos y Atributos

Lo que se puede usar y consultar después de entrenar

Métodos (igual que cualquier modelo)

- `fit(X_train, y_train)` ejecuta la búsqueda · `predict(X)` predice con el mejor modelo encontrado.

Atributo (solo existe después de fit())	Qué contiene
`best_params_`	Diccionario con la mejor combinación
`best_score_`	Score CV promedio de esa combinación
`best_estimator_`	Pipeline ya reentrenado con todo X_train
`cv_results_`	Resultados de TODAS las combinaciones

⚠️ **best_score_ NO es el rendimiento final.** Es un promedio sobre folds de X_train. El rendimiento real se mide con `grid.predict(X_test)`.

El Parámetro scoring: ¿Qué Significa "Mejor"?

GridSearchCV elige la combinación que MAXIMIZA la métrica indicada

scoring	Úsala cuando...	Rango → deseado
`'accuracy'`	Clases balanceadas	0 a 1 → 1
`'precision'`	Un falso positivo es costoso	0 a 1 → 1
`'recall'`	Un falso negativo es costoso (ej. diabetes)	0 a 1 → 1
`'f1'`	Equilibrio entre precision y recall	0 a 1 → 1
`'roc_auc'`	Interesa ordenar casos por riesgo	0.5 (azar) a 1 → 1
`'f1_macro'`	Problemas multiclase	0 a 1 → 1

F1 (usada en el taller)

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

- Otra métrica → otro "mejor modelo".
- Como siempre maximiza, en regresión se usa `neg_mean_squared_error` (error con signo negativo).

Dataset del Taller — Aprobación de Crédito

credito_ecuador.csv · 600 solicitudes · dataset sintético · sin valores nulos

Variable	Tipo	Descripción
`edad`	Numérica	Edad del solicitante (22 a 64 años)
`ingresos_mensuales`	Numérica	Ingresos en USD (400 a 2 432)
`deuda_total`	Numérica	Deuda acumulada en USD (0 a 9 203)
`monto_solicitado`	Numérica	Monto pedido en USD (1 000 a 17 579)
`historial_crediticio`	Binaria	1 = positivo · 0 = negativo
`empleo_estable`	Binaria	1 = estable · 0 = inestable
`aprobado`	Objetivo (y)	1 = aprobado · 0 = rechazado

Balance de clases

- Aprobados: 287 (48%) · Rechazados: 313

Métrica elegida

- Clases balanceadas → `scoring='f1'`

Flujo Completo (1/3): Datos y División

Mismo inicio que en las clases anteriores: construir X, y y separar el conjunto de prueba

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

df = pd.read_csv('credito_ecuador.csv')
X = df.drop('aprobado', axis=1)   # 6 variables predictoras
y = df['aprobado']                # objetivo: 0 o 1

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
```

- `stratify=y` conserva el 48% de aprobados en train y en test.

Flujo Completo (2/3): Modelo Base

Primero un SVM con los valores por defecto: es la línea base para comparar

```
from sklearn.metrics import classification_report

pipe = Pipeline([
    ('escalar', StandardScaler()),   # paso 1
    ('modelo', SVC()),               # paso 2: C=1, rbf
])
pipe.fit(X_train, y_train)           # modelo base
y_pred_base = pipe.predict(X_test)
print(classification_report(y_test, y_pred_base))
```

Resultado real: F1 de la clase Aprobado en prueba = **0.844** (rango 0 a 1 → 1 = deseado).

- Este mismo `pipe` se entrega a GridSearchCV en el siguiente paso.

Flujo Completo (3/3): GridSearchCV

Mismo pipe + param_grid + validación cruzada de 5 folds

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    'modelo__C':      [0.1, 1, 10, 100],
    'modelo__gamma':  [0.001, 0.01, 0.1, 1],
    'modelo__kernel': ['rbf', 'linear'],
}
grid = GridSearchCV(pipe, param_grid, cv=5,
                    scoring='f1', n_jobs=-1)
grid.fit(X_train, y_train)    # 32 × 5 + 1 = 161 modelos
```

- GridSearchCV trabaja con **copias** de `pipe`: el modelo base no cambia.
- `fit()` recibe los datos **sin escalar**: el Pipeline escala dentro de cada fold.

Resultados Reales: Base vs. Optimizado

Conjunto de prueba: 120 solicitudes (57 aprobadas) · métricas de la clase Aprobado

```
print(grid.best_params_)   # C=10, gamma=0.01, 'rbf'
print(grid.best_score_)    # 0.895 (F1 promedio CV)
y_pred_opt = grid.predict(X_test)  # best_estimator_
```

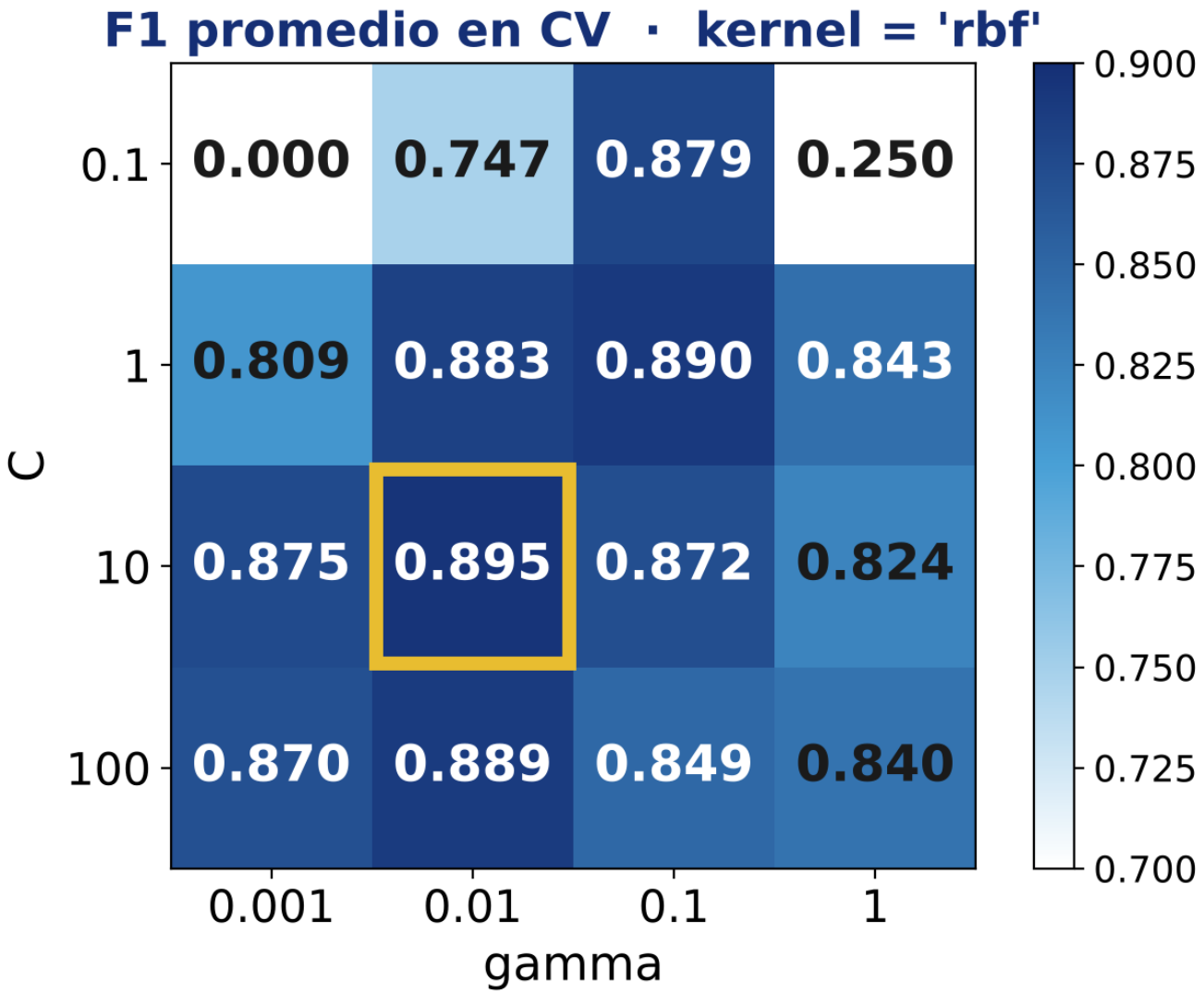
Modelo	Hiperparámetros	Accuracy	Recall	F1
Base	C=1, gamma='scale', rbf	0.858	0.807	0.844
GridSearchCV	C=10, gamma=0.01, rbf	0.883	0.842	0.873

- El modelo optimizado detecta 48 de 57 aprobados (el base, 46).
- Todas las métricas: rango 0 a 1 → 1 = deseado.

+0.029 de F1: los valores por defecto no eran los mejores, y ahora hay evidencia

Visualizar el Espacio de Búsqueda: cv_results_

Mapa de calor real del F1 promedio en CV (kernel = 'rbf')



Cómo leerlo

- Cada celda = F1 promedio de 5 folds.
- Más oscuro = mejor.
- Borde dorado = mejor (0.895).

Lo que muestra

- C = 0.1 y gamma = 0.001: F1 = 0, rechaza a todos.
- Valores extremos de C o gamma bajan el F1.
- Kernel lineal: F1 ≈ 0.88, cerca del RBF.

RandomizedSearchCV: Alternativa más Rápida

Cuando hay demasiadas combinaciones, se prueba solo una muestra al azar

Definición: prueba **n_iter** combinaciones elegidas al azar del espacio, en lugar de todas.

```
from sklearn.model_selection import RandomizedSearchCV

param_dist = {
    'modelo__C': [0.01, 0.1, 1, 10, 100, 1000],
    'modelo__gamma': [0.0001, 0.001, 0.01, 0.1, 1, 10],
    'modelo__kernel': ['rbf', 'linear'],
}
rand = RandomizedSearchCV(pipe, param_dist, n_iter=20, cv=5,
                          scoring='f1', random_state=42)
rand.fit(X_train, y_train)
```

Real: 20 de 72 combinaciones → F1 CV 0.887 (GridSearchCV: 0.895). Cerca, sin garantía de óptimo

Errores Frecuentes y Buenas Prácticas

Lo que hay que revisar antes de confiar en el resultado de la búsqueda

✕ Errores frecuentes

- Escalar antes de GridSearchCV.
- Reportar best_score_ como resultado final.
- Usar X_test para elegir hiperparámetros.
- Usar accuracy con clases desbalanceadas.

✓ Buenas prácticas

- Escala logarítmica para C y gamma.
- Grid grueso primero, luego refinar.
- Si el mejor valor queda en el borde, ampliar el rango.
- Fijar random_state.

Ejemplo real: el mejor (C = 10, gamma = 0.01) quedó en el **interior** del grid. Al ampliar hasta C = 1000, el resultado no cambia: el rango era suficiente.

Entrenamiento Final con el 100% de los Datos

Elegidos los hiperparámetros, el modelo final se entrena con las 600 solicitudes

Modelo de evaluación (80%)

- grid: eligió y midió el modelo.

Pipeline final (100%)

- Se entrena con X, y; se exporta.

```
print(grid.best_params_)   # mejores valores encontrados

pipeline_final = Pipeline([
    ('escalar', StandardScaler()),
    ('modelo', SVC(C=10, gamma=0.01, kernel='rbf')),
])
pipeline_final.fit(X, y)   # X, y completos, no X_train
```

Se copian los valores de best_params_; X_test ya cumplió su función

Exportación del Pipeline con joblib

joblib guarda en un archivo .pkl un objeto de Python ya entrenado

```python
import joblib
# Guarda escalador + modelo juntos, como un solo objeto
joblib.dump(pipeline_final, 'modelo_credito_svm.pkl')
```

¿Por qué exportar el Pipeline completo?

- El archivo incluye la media y desviación del escalador y el SVM con C=10, gamma=0.01.
- Escalado y modelo nunca se desincronizan.
- Otro programa (ej. una API) solo necesita cargar este archivo.

⚠️ **Exportar pipeline_final, no grid.** grid guarda también los resultados de las 32 combinaciones; para producción basta el Pipeline entrenado con el 100% de los datos.

Uso del Modelo con Datos Nuevos

Datos crudos con las mismas 6 columnas: predict() escala por dentro

```
import joblib
import pandas as pd

modelo = joblib.load('modelo_credito_svm.pkl')
nuevos = pd.DataFrame({
    'edad': [35, 52], 'ingresos_mensuales': [1800, 1000],
    'deuda_total': [1200, 8000],
    'monto_solicitado': [6000, 9000],
    'historial_crediticio': [1, 1],
    'empleo_estable': [1, 1]})
print(modelo.predict(nuevos))   # [1 0]: aprobado, rechazado
```

Cliente 1 → 1 (aprobado)

- Ingreso alto, deuda baja.

Cliente 2 → 0 (rechazado)

- Deuda = 8 veces su ingreso.